ПРАКТИКАЛЫҚ ЖҰМЫС №1

Тақырып: Бағдарламалық қосымшаның сапасын талдау және тестілеу талаптарын анықтау

Мақсаты: РО бойынша: білім алушыларда бағдарламалық қосымшаның функционалдық және функционалдық емес талаптарын талдау, бағдарламалық өнімнің сапа сипаттамаларын анықтау, тестілеу мақсаттарын қалыптастыру және тестілеу объектілерін жіктеу дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: бағдарламалық қосымшаның талаптарын талдау, сапа көрсеткіштерін бағалау, тестілеу талаптарын анықтау және бағдарламалық өнімнің сапасын қамтамасыз ету әдістерін қолдану дағдыларын игер

Әдістемелік нұсқаулық

1. Бағдарламалық қосымшаны таңдаңыз.
2. Қосымшаның негізгі мақсатын анықтаңыз.
3. Функционалдық талаптарды тізімдеңіз.
4. Функционалдық емес талаптарды анықтаңыз.
5. Талаптарды маңыздылығы бойынша жіктеңіз.
6. Бағдарламалық өнімнің пайдаланушыларын анықтаңыз.
7. Негізгі пайдаланушы сценарийлерін құрастырыңыз.
8. Әр талап үшін тексерілетін шарттарды белгілеңіз.
9. Бағдарламалық өнімнің сапа сипаттамаларын анықтаңыз.
10. ISO/IEC 25010 сапа моделін қолданыңыз.
11. Тестілеуді қажет ететін функционалдарды анықтаңыз.
12. Тәуекелдерді анықтаңыз.
13. Тәуекелдерді ықтималдығы мен әсеріне қарай бағалаңыз.
14. Тестілеу басымдықтарын белгілеңіз.
15. Талап пен тест арасындағы байланысты көрсетіңіз.
16. Тестілік жағдайлардың бастапқы тізімін жасаңыз.
17. Талаптардың толықтығын тексеріңіз.
18. Қайшылықты және түсініксіз талаптарды табыңыз.
19. Талаптарды тестілеуге жарамдылығы бойынша бағалаңыз.
20. Нәтижелерді кесте түрінде рәсімдеңіз.

Жабдықтау

- Дербес компьютер;
- Интернет желісі;
- Windows/Linux/macOS операциялық жүйесі;
- Visual Studio Code немесе PyCharm;
- Python 3.x;
- Git және GitHub;
- Microsoft Word немесе Google Docs;
- Microsoft Excel немесе Google Sheets;
- draw.io немесе diagrams.net;
- браузер: Google Chrome, Mozilla Firefox немесе Microsoft Edge;
- ISO/IEC 25010 сапа моделі бойынша ақпараттық материалдар;
- пән бойынша дәріс материалдары.

Жұмыс барысы

1-кезең. Бағдарламалық қосымшаны таңдау

Практикалық жұмысты орындау үшін төмендегі қосымшалардың бірін таңдаңыз:

- интернет-дүкен;
- университеттің ақпараттық жүйесі;
- кітапхана жүйесі;

- қонақүйді брондау жүйесі;
- медициналық ақпараттық жүйе;
- банк қосымшасы;
- электрондық журнал;
- жеткізу қызметі;
- билет сатып алу жүйесі;
- студенттерді тіркеу жүйесі.

Тапсырмалар

Тапсырма 1. Қосымшаны сипаттау

Таңдалған бағдарламалық қосымшаның атауын, мақсатын және негізгі қызметтерін сипаттаңыз.

Нәтиже: 5–7 сөйлемнен тұратын сипаттама.

Тапсырма 2. Пайдаланушыларды анықтау

Қосымшамен жұмыс істейтін пайдаланушылардың негізгі категорияларын анықтаңыз.

Мысалы:

Пайдаланушы	**Рөлі**
Әкімші	Жүйені басқарады
Студент	Ақпаратты қарайды
Оқытушы	Баға енгізеді

Тапсырма 3. Функционалдық талаптарды анықтау

Қосымшаның кемінде **10 функционалдық талабын** анықтаңыз.

Мысалы:

Жүйе пайдаланушыға логин және пароль арқылы авторизациялануға мүмкіндік беруі тиіс.

Тапсырма 4. Функционалдық емес талаптарды анықтау

Кемінде **10 функционалдық емес талап** анықтаңыз.

Мысалы:

- өнімділік;
- қауіпсіздік;
- сенімділік;
- қолдануға ыңғайлылық;
- масштабталу;
- қолжетімділік.

Тапсырма 5. Талаптарды жіктеу

Барлық талаптарды функционалдық және функционалдық емес талаптарға бөліңіз.

№	**Талап**	**Түрі**	**Басымдығы**
1	Авторизация	Функционалдық	Жоғары
2	Жауап беру уақыты ≤ 2 с	Функционалдық емес	Жоғары

Тапсырма 6. ISO/IEC 25010 сапа сипаттамаларын анықтау

Таңдалған қосымша үшін ISO/IEC 25010 моделінің кемінде **8 сапа сипаттамасын** анықтаңыз.

Мысалы:

- функционалдық жарамдылық;
- өнімділік;
- үйлесімділік;
- қолдануға жарамдылық;
- сенімділік;
- қауіпсіздік;
- сүйемелдеуге жарамдылық;

- тасымалданғыштық.

Тапсырма 7. Пайдаланушы сценарийін құру

Бір негізгі функция үшін пайдаланушы сценарийін жасаңыз.

Мысал:

Жүйеге кіру → логин енгізу → пароль енгізу → «Кіру» батырмасын басу → жүйені тексеру → пайдаланушы кабинетін ашу.

Тапсырма 8. Use Case анықтау

Таңдалған қосымша үшін кемінде **5 Use Case** анықтаңыз.

Мысалы:

1. Login;
2. Registration;
3. Search;
4. Create record;
5. Delete record.

Тапсырма 9. Талаптардың тексерілу мүмкіндігін анықтау

Әр талаптың тест арқылы тексерілу мүмкіндігін анықтаңыз.

Талап	Тексеруге болады ма?	Тексеру әдісі
Жүйеге кіру	Иә	Functional testing
Жүйе ыңғайлы болуы тиіс	Толық емес	Usability testing

Тапсырма 10. Түсініксіз талаптарды табу

Берілген талаптардан түсініксіз немесе өлшенбейтін талаптарды анықтаңыз:

1. Жүйе жылдам жұмыс істеуі тиіс.
2. Интерфейс ыңғайлы болуы керек.
3. Қосымша сенімді болуы тиіс.
4. Жүйе көп пайдаланушыға қызмет көрсетуі керек.
5. Ақпарат тез жүктелуі тиіс.

Әр талапты **өлшенетін талапқа** айналдырыңыз.

Тапсырма 11. Талаптардағы қайшылықтарды анықтау

Кемінде 3 қарама-қайшы талап мысалын жасаңыз.

Мысалы:

«Пайдаланушы кез келген уақытта парольді өзгерте алады».

және

«Парольді өзгерту тек әкімшінің рұқсатымен орындалады».

Қайшылықты шешудің нұсқасын ұсыныңыз.

Тапсырма 12. Талаптардың басымдығын анықтау

Талаптарды:

- Critical;
- High;
- Medium;
- Low

деңгейлеріне бөліңіз.

Кемінде **5 талап** пайдаланыңыз.

Тапсырма 13. Тестілеу объектілерін анықтау

Қосымшада тестіленуі тиіс кемінде **10 объектіні** анықтаңыз.

Мысалы:

- Login;
- Password;
- Registration;
- Search;
- Database;
- API;

- User interface;
- Reports.

Тапсырма 14. Тәуекелдерді анықтау

Бағдарламалық қосымша үшін кемінде **5 тәуекелді** анықтаңыз.

№	Тәуекел	Ықтималдық	Әсері	Деңгей
1	Деректер жоғалуы	Жоғары	Жоғары	Critical
2	API қатесі	Орташа	Жоғары	High

Тапсырма 15. Тәуекел матрицасын құру

Тәуекелдерді ықтималдық және әсер көрсеткіштері бойынша орналастырыңыз.

Risk = Probability × Impact

Әр тәуекелге 1–5 аралығында балл беріңіз.

Тапсырма 16. Тестілеу басымдықтарын анықтау

Тәуекел деңгейіне байланысты тестілеу басымдықтарын белгілеңіз.

Мысалы:

Critical → бірінші кезекте тестілеу

High → жоғары басымдық

Medium → жоспар бойынша

Low → соңғы кезеңде

Тапсырма 17. Тест сценарийлерінің бастапқы тізімін құру

Кемінде **15 тест сценарийін** жасаңыз.

ID	Тест сценарийі	Күтілетін нәтиже	Басымдық
TS-01	Дұрыс логинмен кіру	Жүйеге кіреді	High
TS-02	Қате пароль енгізу	Қате туралы хабарлама	High

Тапсырма 18. Requirements Traceability Matrix құру

Талаптарды тест сценарийлерімен байланыстырыңыз.

Requirement ID	Requirement	Test ID	Status
REQ-01	Авторизация	TS-01	Planned
REQ-02	Қате пароль	TS-02	Planned

Тапсырма 19. Тестілеу стратегиясын қалыптастыру

Таңдалған қосымша үшін қысқаша тестілеу стратегиясын жасаңыз.

Стратегияда:

- не тестіленеді;
- не тестіленбейді;
- қандай тест түрлері қолданылады;
- қандай құралдар пайдаланылады;
- қандай тәуекелдер басым;
- тестілеу нәтижесі қалай бағаланатынын

көрсетіңіз.

Тапсырма 20. Талдау нәтижесі бойынша қорытынды жасау

Барлық жүргізілген талдаулардың нәтижесінде бағдарламалық қосымшаның сапасына бастапқы баға беріңіз.

Қорытындыда:

1. негізгі тәуекелдерді;
2. негізгі тестілеу объектілерін;
3. ең маңызды талаптарды;
4. ұсынылатын тестілеу түрлерін;
5. автоматтандыруға болатын тестілерді

көрсетіңіз.

Қорытынды

Практикалық жұмыс нәтижесінде білім алушы:

- бағдарламалық қосымшаның талаптарын талдау;
- функционалдық және функционалдық емес талаптарды ажырату;
- бағдарламалық өнім сапасының сипаттамаларын анықтау;
- тестілеу объектілерін белгілеу;
- тәуекелдерді бағалау;
- тест сценарийлерін қалыптастыру;
- Requirements Traceability Matrix құру;
- тестілеу стратегиясын әзірлеу

дағдыларын қалыптастырады.

Есеп құрамында: қосымшаның сипаттамасы, талаптар кестесі, сапа сипаттамалары, Use Case, тәуекелдер матрицасы, тест сценарийлері, Traceability Matrix және қорытынды болуы тиіс.

Бақылау сұрақтары

1. Бағдарламалық қамтамасыз етудің сапасы дегеніміз не?
2. Функционалдық талап дегеніміз не?
3. Функционалдық емес талап дегеніміз не?
4. Функционалдық және функционалдық емес талаптардың айырмашылығы қандай?
5. Software Quality қандай негізгі сипаттамалардан тұрады?
6. ISO/IEC 25010 стандарты не үшін қолданылады?
7. Тестілеу объектісі дегеніміз не?
8. Test Scenario мен Test Case айырмашылығы қандай?
9. Тестілеу стратегиясы дегеніміз не?
10. Тестілеу басымдығы қалай анықталады?
11. Software Testing Life Cycle дегеніміз не?
12. Тәуекел дегеніміз не?
13. Risk = Probability × Impact формуласын түсіндіріңіз.
14. Critical және Low тәуекелдердің айырмашылығы қандай?
15. Requirements Traceability Matrix не үшін қажет?
16. Қандай талаптарды автоматты түрде тестілеуге болады?
17. Қандай талаптар тестілеуге қиын?
18. Өлшенбейтін талаптарды қалай нақтылауға болады?
19. Талаптар арасындағы қайшылықтарды қалай анықтауға болады?
20. Талаптарды талдау не үшін тестілеудің бірінші кезеңдерінің бірі болып табылады?

Бағалау критерийлері

Жалпы – 100 балл

Критерий	Балл
1–5 тапсырмаларды орындау	20
6–10 тапсырмаларды орындау	20
11–15 тапсырмаларды орындау	20
16–20 тапсырмаларды орындау	20
Есепті рәсімдеу	10
Талдау және қорытынды сапасы	10
Барлығы	**100**

Бағалау деңгейлері
90–100 балл: барлық тапсырмалар толық және дұрыс орындалған, талаптар терең талданған, тест сценарийлері негізделген, қорытынды сапалы.
75–89 балл: тапсырмалардың көпшілігі дұрыс орындалған, жекелеген кемшіліктер бар.
50–74 балл: негізгі тапсырмалар орындалған, бірақ талдау мен негіздеу жеткіліксіз.
0–49 балл: тапсырмалардың едәуір бөлігі орындалмаған немесе нәтижелер талапқа сәйкес емес.

Негізгі әдебиеттер

1. Myers G. J., Sandler C., Badgett T. **The Art of Software Testing.** 4th ed. Wiley.
2. Ammann P., Offutt J. **Introduction to Software Testing.** Cambridge University Press.
3. Graham D., van Veenendaal E., Evans I., Black R. **Foundations of Software Testing.**
4. Spillner A., Linz T., Schaefer H. **Software Testing Foundations.**
5. Meszaros G. **xUnit Test Patterns: Refactoring Test Code.** Addison-Wesley.

Қосымша әдебиеттер және онлайн ресурстар

6. ISTQB. **Certified Tester Foundation Level Syllabus.**
7. ISO/IEC 25010. **Systems and software Quality Requirements and Evaluation (SQuaRE).**
8. OWASP. **Web Security Testing Guide.**
9. OWASP. **OWASP Top 10.**
10. Fowler M. **Refactoring.** Addison-Wesley.
11. Martin R. C. **Clean Code.** Prentice Hall.
12. Beck K. **Test-Driven Development: By Example.** Addison-Wesley.
13. pytest Documentation — https://docs.pytest.org/
14. Selenium Documentation — https://www.selenium.dev/documentation/
15. Postman Learning Center — https://learning.postman.com/
16. SonarQube Documentation — https://docs.sonarsource.com/sonarqube/
17. GitHub Actions Documentation — https://docs.github.com/actions
18. OWASP ZAP — https://www.zaproxy.org/
19. Apache JMeter — https://jmeter.apache.org/
20. Playwright Documentation — https://playwright.dev/

ПРАКТИКАЛЫҚ ЖҰМЫС №2

Тақырып: Тестілеу стратегиясы және Software Testing Life Cycle (STLC)

Мақсаты: РО бойынша: бағдарламалық қосымшаны тестілеуді жоспарлау, тестілеу стратегиясын әзірлеу, STLC кезеңдерін анықтау және тестілеу ресурстарын жоспарлау дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: бағдарламалық өнімнің сапасын қамтамасыз ету үшін тестілеу процесін ұйымдастыру, тестілеу әдістерін таңдау және тестілеу құжаттамасын әзірлеу дағдыларын игеру.

Әдістемелік нұсқаулық

1. Тестіленетін қосымшаны таңдаңыз.
2. Қосымшаның тестілеу мақсаттарын анықтаңыз.
3. Тестілеу объектілерін белгілеңіз.
4. STLC кезеңдерін анықтаңыз.
5. Әр кезеңнің міндеттерін тізімдеңіз.
6. Тестілеу стратегиясын жасаңыз.
7. Тестілеу көлемін анықтаңыз.

8. Scope In және Scope Out элементтерін белгілеңіз.
9. Тестілеу деңгейлерін таңдаңыз.
10. Тестілеу түрлерін таңдаңыз.
11. Тестілеу ортасын анықтаңыз.
12. Тестілік деректерді анықтаңыз.
13. Тестілеу ресурстарын жоспарлаңыз.
14. Тәуекелдерді анықтаңыз.
15. Exit Criteria белгілеңіз.
16. Entry Criteria белгілеңіз.
17. Test Deliverables тізімін жасаңыз.
18. Тестілеу кестесін құрыңыз.
19. Тестілеу нәтижелерін бағалау метрикаларын анықтаңыз.
20. Test Plan құжатын рәсімдеңіз.

Жабдықтау

Компьютер, Visual Studio Code/PyCharm, Git/GitHub, Excel/Google Sheets, Word/Google Docs, draw.io, таңдалған бағдарламалық қосымша.

Жұмыс барысы

STLC кезеңдерін талдап, таңдалған қосымша үшін толық тестілеу жоспарын құрыңыз.

Тапсырмалар

1. Тестіленетін қосымшаны таңдаңыз.
2. Тестілеудің мақсатын жазыңыз.
3. STLC кезеңдерін тізімдеңіз.
4. Әр кезеңнің мақсатын анықтаңыз.
5. Test Planning кезеңін сипаттаңыз.
6. Requirement Analysis кезеңін сипаттаңыз.
7. Test Design кезеңін сипаттаңыз.
8. Test Environment Setup кезеңін сипаттаңыз.
9. Test Execution кезеңін сипаттаңыз.
10. Test Closure кезеңін сипаттаңыз.
11. Scope In анықтаңыз.
12. Scope Out анықтаңыз.
13. Entry Criteria құрыңыз.
14. Exit Criteria құрыңыз.
15. Тестілеу түрлерін таңдаңыз.
16. Тестілеу құралдарын таңдаңыз.
17. Тестілеу тәуекелдерін анықтаңыз.
18. Тестілеу кестесін құрыңыз.
19. Test Deliverables тізімін жасаңыз.
20. Қорытынды Test Plan құжатын әзірлеңіз.

Қорытынды

STLC кезеңдері көрсетілген **Test Plan** құжатын, тестілеу стратегиясын және тестілеу кестесін ұсыныңыз.

Бақылау сұрақтары

1. STLC дегеніміз не?
2. STLC қандай кезеңдерден тұрады?
3. Test Plan дегеніміз не?
4. Test Strategy дегеніміз не?
5. Test Plan мен Test Strategy айырмашылығы қандай?
6. Entry Criteria дегеніміз не?
7. Exit Criteria дегеніміз не?
8. Scope In дегеніміз не?
9. Scope Out дегеніміз не?
10. Test Deliverable дегеніміз не?
11. Тестілеу тәуекелі дегеніміз не?
12. Тестілеу ортасы дегеніміз не?
13. Test Execution кезеңінің мақсаты қандай?
14. Test Closure қашан орындалады?
15. Тестілеу ресурстары қалай анықталады?
16. Тестілеу кестесі не үшін қажет?
17. Тестілеу стратегиясын кім әзірлейді?
18. Agile ортада тестілеу қалай ұйымдастырылады?
19. Continuous Testing дегеніміз не?
20. STLC пен SDLC арасында қандай байланыс бар?

Бағалау критерийлері

Критерий	Балл
1–5 тапсырмалар	20
6–10 тапсырмалар	20
11–15 тапсырмалар	20
16–20 тапсырмалар	20
Есепті рәсімдеу	10
Талдау және қорытынды	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Myers G. J., Sandler C., Badgett T. *The Art of Software Testing.*
2. Ammann P., Offutt J. *Introduction to Software Testing.*
3. ISTQB. *Certified Tester Foundation Level Syllabus.*
4. Graham D. et al. *Foundations of Software Testing.*
5. ISO/IEC 25010.
6. https://www.istqb.org/
7. https://docs.pytest.org/

ПРАКТИКАЛЫҚ ЖҰМЫС №3

Тақырып: Тестілеу деңгейлері: Unit, Integration, System және Acceptance Testing

Мақсаты:РО бойынша: бағдарламалық жүйенің тестілеу деңгейлерін ажырату және әр деңгейге сәйкес тестілерді жобалау дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: бағдарламалық жүйенің компоненттерін жеке және интеграцияланған түрде тексеру, жүйелік және қабылдау тестілерін ұйымдастыру дағдыларын игеру.

Әдістемелік нұсқаулық

1. Бағдарламаны компоненттерге бөліңіз.
2. Әр компоненттің функциясын анықтаңыз.
3. Unit Testing объектілерін белгілеңіз.
4. Integration Testing байланыстарын анықтаңыз.
5. System Testing сценарийлерін жасаңыз.
6. Acceptance критерийлерін анықтаңыз.
7. Әр деңгейге тест-кейс жасаңыз.
8. Күтілетін нәтижелерді анықтаңыз.
9. Нақты нәтижелерді тіркеңіз.
10. Тест нәтижелерін салыстырыңыз.
11. Қателерді тіркеңіз.
12. Regression тестін орындаңыз.
13. Тестілеу деңгейлерін салыстырыңыз.
14. Автоматтандыру мүмкіндігін анықтаңыз.
15. Нәтижелерді кестеге енгізіңіз.
16. Coverage көрсеткішін анықтаңыз.
17. Ақаулардың таралуын талдаңыз.
18. Қай деңгейде қандай қате табылғанын анықтаңыз.
19. Тестілеу пирамидасын құрыңыз.
20. Қорытынды есепті рәсімдеңіз.

Жабдықтау

Python, pytest/unittest, IDE, Git, Excel, Postman, браузер.

Жұмыс барысы

Таңдалған қосымшаны төрт тестілеу деңгейі бойынша зерттеңіз.

Тапсырмалар

1. Қосымшаның архитектурасын анықтаңыз.
2. Компоненттерді тізімдеңіз.
3. Unit Test объектілерін анықтаңыз.
4. 5 Unit Test сценарийін жасаңыз.
5. Unit Test орындаңыз.
6. Екі компоненттің байланысын анықтаңыз.
7. Integration Test сценарийлерін жасаңыз.
8. API интеграциясын тексеріңіз.
9. System Test сценарийін жасаңыз.
10. End-to-End сценарий жасаңыз.
11. Acceptance критерийлерін анықтаңыз.
12. Acceptance Test жасаңыз.
13. Төрт тестілеу деңгейін салыстырыңыз.
14. Әр деңгейдегі тест санын анықтаңыз.
15. Тестілеу нәтижелерін тіркеңіз.

16. Анықталған defect тізімін жасаңыз.
17. Defect severity белгілеңіз.
18. Test Coverage есептеңіз.
19. Testing Pyramid схемасын жасаңыз.
20. Нәтижелер бойынша қорытынды жасаңыз.

Қорытынды

Unit, Integration, System және Acceptance тестілерінің нәтижелерін салыстыратын есеп дайындаңыз.

Бақылау сұрақтары

1. Unit Testing дегеніміз не?
2. Integration Testing дегеніміз не?
3. System Testing дегеніміз не?
4. Acceptance Testing дегеніміз не?
5. Unit Test объектісі қандай?
6. Integration Testing мақсаты қандай?
7. System Testing ерекшелігі қандай?
8. Acceptance критерийі дегеніміз не?
9. End-to-End Testing дегеніміз не?
10. Test Pyramid дегеніміз не?
11. Unit Test артықшылығы қандай?
12. Integration Test қандай ақауларды анықтайды?
13. System Testing кіммен орындалады?
14. Acceptance Testing кім үшін қажет?
15. Test Coverage дегеніміз не?
16. Regression Testing қай кезде орындалады?
17. Тестілеу деңгейлерінің байланысы қандай?
18. Unit Test автоматтандырылуы мүмкін бе?
19. Integration Testing кезінде қандай мәселелер кездеседі?
20. Төрт деңгейді бір жүйеде қалай қолдануға болады?

Бағалау критерийлері

Критерий	Балл
1–5 тапсырмалар	20
6–10 тапсырмалар	20
11–15 тапсырмалар	20
16–20 тапсырмалар	20
Есепті рәсімдеу	10
Талдау және қорытынды	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Ammann P., Offutt J. *Introduction to Software Testing.*
2. Myers G. *The Art of Software Testing.*
3. Meszaros G. *xUnit Test Patterns.*

4. ISTQB Foundation Level.
5. pytest Documentation.
6. unittest Documentation.

ПРАКТИКАЛЫҚ ЖҰМЫС №4

Тақырып: Black-box және White-box тестілеу әдістері

Мақсаты: РО бойынша: бағдарламалық жүйені Black-box және White-box тәсілдері арқылы тестілеу әдістерін қолдану дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: тестілік деректерді таңдау, бағдарламаның ішкі құрылымын талдау және тестілеудің тиімді әдістерін таңдау дағдыларын игеру.

Әдістемелік нұсқаулық

1. Функционалдық талаптарды анықтаңыз.
2. Black-box объектілерін белгілеңіз.
3. Equivalence Partitioning қолданыңыз.
4. Boundary Value Analysis қолданыңыз.
5. Decision Table жасаңыз.
6. State Transition диаграммасын құрыңыз.
7. White-box кодын таңдаңыз.
8. Control Flow Graph құрыңыз.
9. Branch Coverage анықтаңыз.
10. Statement Coverage есептеңіз.
11. Тест деректерін таңдаңыз.
12. Тестілерді орындаңыз.
13. Нәтижелерді тіркеңіз.
14. Қателерді анықтаңыз.
15. Severity белгілеңіз.
16. Coverage көрсеткішін есептеңіз.
17. Әдістерді салыстырыңыз.
18. Артықшылықтарын анықтаңыз.
19. Кемшіліктерін анықтаңыз.
20. Қорытынды жасаңыз.

Жабдықтау

Python, IDE, pytest, Excel, draw.io, Git.

Тапсырмалар

1. Black-box тестілеу объектісін таңдаңыз.
2. 5 эквиваленттік класс құрыңыз.
3. Boundary Value Analysis орындаңыз.
4. 10 шекаралық тест жасаңыз.
5. Decision Table құрыңыз.
6. Decision Table бойынша тест жасаңыз.
7. State Transition диаграммасын жасаңыз.
8. State Transition тестілерін жасаңыз.
9. White-box үшін код таңдаңыз.

10. Control Flow Graph құрыңыз.
11. Statement Coverage есептеңіз.
12. Branch Coverage есептеңіз.
13. Path Coverage талдаңыз.
14. Cyclomatic Complexity анықтаңыз.
15. Негізгі тест жолдарын анықтаңыз.
16. Тестілерді орындаңыз.
17. Black-box және White-box нәтижелерін салыстырыңыз.
18. Анықталған defect тізімін жасаңыз.
19. Әдістердің тиімділігін бағалаңыз.
20. Есепті рәсімдеңіз.

Қорытынды

Black-box және White-box әдістерінің нәтижелерін салыстырып, қандай жағдайда қай әдіс тиімді екенін негіздеңіз.

Бақылау сұрақтары

1. Black-box Testing дегеніміз не?
2. White-box Testing дегеніміз не?
3. Equivalence Partitioning дегеніміз не?
4. Boundary Value Analysis дегеніміз не?
5. Decision Table не үшін қолданылады?
6. State Transition Testing дегеніміз не?
7. Control Flow Graph дегеніміз не?
8. Statement Coverage дегеніміз не?
9. Branch Coverage дегеніміз не?
10. Path Coverage дегеніміз не?
11. Cyclomatic Complexity дегеніміз не?
12. Оның формуласы қандай?
13. Black-box артықшылығы қандай?
14. White-box артықшылығы қандай?
15. Қай кезде Black-box қолданылады?
16. Қай кезде White-box тиімді?
17. Boundary Value Analysis неге маңызды?
18. Coverage 100% болса, бағдарлама қатесіз деп айтуға бола ма?
19. Decision Table қандай есептерге тиімді?
20. Екі әдісті бірге қолданудың артықшылығы қандай?

Бағалау критерийлері

Критерий	Балл
1–5 тапсырмалар	20
6–10 тапсырмалар	20
11–15 тапсырмалар	20
16–20 тапсырмалар	20
Рәсімдеу	10
Қорытынды	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Ammann P., Offutt J. *Introduction to Software Testing.*
2. Myers G. *The Art of Software Testing.*
3. ISTQB Foundation Level.
4. Pressman R. *Software Engineering.*
5. Python pytest Documentation.

ПРАКТИКАЛЫҚ ЖҰМЫС №5

Тақырып: Test Case және Test Scenario жобалау

Мақсаты:РО бойынша: тестілік сценарийлерді, тест-кейстерді және күтілетін нәтижелерді жобалау дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: тестілік құжаттаманы әзірлеу, тест жағдайларын басымдық бойынша жіктеу және тест нәтижелерін рәсімдеу дағдыларын игеру.

Әдістемелік нұсқаулық

1. Тестіленетін функцияны таңдаңыз.
2. Талаптарды талдаңыз.
3. Test Scenario құрыңыз.
4. Test Case ID беріңіз.
5. Preconditions анықтаңыз.
6. Test Data дайындаңыз.
7. Test Steps жазыңыз.
8. Expected Result анықтаңыз.
9. Priority белгілеңіз.
10. Severity анықтаңыз.
11. Positive Test жасаңыз.
12. Negative Test жасаңыз.
13. Boundary Test жасаңыз.
14. Test Case орындаңыз.
15. Actual Result тіркеңіз.
16. Status белгілеңіз.
17. Defect Report жасаңыз.
18. Test Case қайта орындаңыз.
19. Traceability жасаңыз.
20. Есепті рәсімдеңіз.

Жабдықтау

Excel/Google Sheets, Word, TestRail аналогтары, Jira немесе GitHub Issues, Postman, браузер.

Тапсырмалар

1. Тестіленетін функцияны таңдаңыз.
2. 5 Test Scenario жасаңыз.
3. 10 Positive Test Case жасаңыз.
4. 10 Negative Test Case жасаңыз.

5. Preconditions анықтаңыз.
6. Test Data жасаңыз.
7. Test Steps жазыңыз.
8. Expected Result анықтаңыз.
9. Priority тағайындаңыз.
10. Severity тағайындаңыз.
11. Positive және Negative тестілерді салыстырыңыз.
12. Boundary тестілер жасаңыз.
13. Тестілерді орындаңыз.
14. Actual Result енгізіңіз.
15. Status белгілеңіз.
16. Failed Test Case анықтаңыз.
17. Defect Report жасаңыз.
18. Retest орындаңыз.
19. Requirements Traceability Matrix құрыңыз.
20. Test Case жинағын рәсімдеңіз.

Қорытынды

Кемінде 20 тест-кейстен тұратын тестілік құжаттама дайындаңыз.

Бақылау сұрақтары

1. Test Case дегеніміз не?
2. Test Scenario дегеніміз не?
3. Test Case пен Scenario айырмашылығы қандай?
4. Preconditions дегеніміз не?
5. Test Data дегеніміз не?
6. Expected Result дегеніміз не?
7. Actual Result дегеніміз не?
8. Priority дегеніміз не?
9. Severity дегеніміз не?
10. Positive Testing дегеніміз не?
11. Negative Testing дегеніміз не?
12. Failed Test Case дегеніміз не?
13. Retesting дегеніміз не?
14. Regression Testing дегеніміз не?
15. Test Case ID не үшін қажет?
16. Traceability Matrix не үшін қажет?
17. Test Case қалай құрылымдалады?
18. Жақсы Test Case қандай болуы керек?
19. Test Case автоматтандырылуы мүмкін бе?
20. Тест-кейстерді қалай басымдыққа бөлуге болады?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20

Критерий	**Балл**
Рәсімдеу	10
Талдау	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Graham D. et al. *Foundations of Software Testing.*
2. ISTQB Foundation Level.
3. Myers G. *The Art of Software Testing.*
4. Kaner C. et al. *Lessons Learned in Software Testing.*

ПРАКТИКАЛЫҚ ЖҰМЫС №6

Тақырып: Unit Testing және автоматтандырылған тестілеу

Мақсаты:

РО бойынша: бағдарламалық модульдер үшін unit-тестілер әзірлеу және автоматты түрде орындау дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: бағдарламалық кодтың дұрыстығын автоматты тестілеу құралдарымен тексеру және тест нәтижелерін талдау дағдыларын игеру.

Әдістемелік нұсқаулық

1. Бағдарламалық модульді таңдаңыз.
2. Функциялардың тізімін жасаңыз.
3. Кіріс параметрлерін анықтаңыз.
4. Шығыс нәтижелерін анықтаңыз.
5. pytest орнатыңыз.
6. Test File жасаңыз.
7. Test Function құрыңыз.
8. Assert қолданыңыз.
9. Positive Test жасаңыз.
10. Negative Test жасаңыз.
11. Edge Case жасаңыз.
12. Exception Test жасаңыз.
13. Parametrization қолданыңыз.
14. Fixture жасаңыз.
15. Тестілерді іске қосыңыз.
16. Failed тестілерді анықтаңыз.
17. Кодты түзетіңіз.
18. Тестілерді қайта орындаңыз.
19. Coverage есептеңіз.
20. Нәтижелерді рәсімдеңіз.

Жабдықтау

Python 3.x, pytest, pytest-cov, VS Code/PyCharm, Git.

Тапсырмалар

1. Python жобасын жасаңыз.
2. Тестілейтін модуль жасаңыз.
3. 5 функция жазыңыз.
4. Әр функцияға Unit Test жасаңыз.
5. Positive Test жасаңыз.
6. Negative Test жасаңыз.
7. Edge Case жасаңыз.
8. Exception тест жасаңыз.
9. assert қолданыңыз.
10. pytest.raises() қолданыңыз.
11. Parametrized Test жасаңыз.
12. Fixture жасаңыз.
13. Test Suite жасаңыз.
14. Тестілерді орындаңыз.
15. Failed тестіні анықтаңыз.
16. Қате кодты түзетіңіз.
17. Retest орындаңыз.
18. pytest-cov арқылы Coverage есептеңіз.
19. Coverage нәтижесін талдаңыз.
20. Unit Testing есебін дайындаңыз.

Қорытынды

Автоматтандырылған Unit Test Suite және Coverage есебін ұсыныңыз.

Бақылау сұрақтары

1. Unit Testing дегеніміз не?
2. Unit Test қандай мақсатта қолданылады?
3. pytest дегеніміз не?
4. unittest дегеніміз не?
5. assert не үшін қажет?
6. Fixture дегеніміз не?
7. Parametrization дегеніміз не?
8. Mock дегеніміз не?
9. Exception Testing дегеніміз не?
10. Test Suite дегеніміз не?
11. Code Coverage дегеніміз не?
12. Coverage түрлері қандай?
13. 100% Coverage нені білдіреді?
14. Unit Test қандай болуы керек?
15. Unit Test артықшылығы қандай?
16. Автоматтандырудың пайдасы қандай?
17. Failed Test қалай талданады?
18. Retest не үшін орындалады?
19. Unit Test CI/CD-ге қалай енгізіледі?
20. Unit Testing пен Integration Testing айырмашылығы қандай?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20
Код сапасы	10
Есеп және қорытынды	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Meszaros G. *xUnit Test Patterns.*
2. Beck K. *Test-Driven Development: By Example.*
3. pytest Documentation.
4. Python unittest Documentation.
5. ISTQB Foundation Level.

ПРАКТИКАЛЫҚ ЖҰМЫС №7

Тақырып: Debugging – бағдарламалық қателерді анықтау және түзету

Мақсаты:РО бойынша: бағдарламалық кодтағы қателерді жүйелі түрде анықтау, себептерін талдау және түзету дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: debugger, logging және диагностикалық құралдарды пайдаланып бағдарламалық ақауларды анықтау және түзету дағдыларын игеру.

Әдістемелік нұсқаулық

1. Қате коды бар бағдарламаны таңдаңыз.
2. Қатені қайта шығарыңыз.
3. Error Message анықтаңыз.
4. Stack Trace талдаңыз.
5. Қате орналасқан жолды табыңыз.
6. Breakpoint орнатыңыз.
7. Бағдарламаны Debug режимінде іске қосыңыз.
8. Айнымалыларды бақылаңыз.
9. Step Over қолданыңыз.
10. Step Into қолданыңыз.
11. Step Out қолданыңыз.
12. Call Stack талдаңыз.
13. Logging енгізіңіз.
14. Қате себебін анықтаңыз.
15. Кодты түзетіңіз.
16. Тестті қайта орындаңыз.
17. Regression Test орындаңыз.
18. Түзету нәтижесін салыстырыңыз.
19. Defect Report жаңартыңыз.

20. Debugging есебін рәсімдеңіз.

Жабдықтау

VS Code/PyCharm/Visual Studio, Python/Java/C#, debugger, Git.

Тапсырмалар

1. Қате бағдарламаны таңдаңыз.
2. Қатені іске қосыңыз.
3. Error Message тіркеңіз.
4. Stack Trace сақтаңыз.
5. Қате жолын анықтаңыз.
6. Breakpoint орнатыңыз.
7. Debug режимін іске қосыңыз.
8. Variable мәндерін бақылаңыз.
9. Step Over қолданыңыз.
10. Step Into қолданыңыз.
11. Step Out қолданыңыз.
12. Call Stack зерттеңіз.
13. Logging қосыңыз.
14. Root Cause анықтаңыз.
15. Қате кодты түзетіңіз.
16. Unit Test орындаңыз.
17. Regression Test орындаңыз.
18. Before/After нәтижесін салыстырыңыз.
19. Debugging журналын жасаңыз.
20. Қорытынды есеп дайындаңыз.

Қорытынды

Анықталған қателердің **Symptom → Cause → Fix → Retest** тізбегін көрсететін есеп дайындаңыз.

Бақылау сұрақтары

1. Debugging дегеніміз не?
2. Bug дегеніміз не?
3. Error мен Failure айырмашылығы қандай?
4. Stack Trace дегеніміз не?
5. Breakpoint не үшін қолданылады?
6. Step Over дегеніміз не?
7. Step Into дегеніміз не?
8. Step Out дегеніміз не?
9. Call Stack дегеніміз не?
10. Logging не үшін қажет?
11. Root Cause дегеніміз не?
12. Debugger қандай мүмкіндіктер береді?
13. Syntax Error дегеніміз не?
14. Runtime Error дегеніміз не?
15. Logical Error дегеніміз не?
16. Debugging пен Testing айырмашылығы қандай?

17. Regression Testing не үшін қажет?
18. Debugging процесін қалай жүйелеуге болады?
19. Қатені түзеткеннен кейін не істеу керек?
20. Debugging кезінде қандай қателер жиі кездеседі?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20
Практикалық debugging	10
Есеп	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Martin R. C. *Clean Code.*
2. Fowler M. *Refactoring.*
3. Python Debugging Documentation.
4. Microsoft Visual Studio Debugger Documentation.
5. VS Code Debugging Documentation.

ПРАКТИКАЛЫҚ ЖҰМЫС №8

Тақырып: Static Code Analysis және Code Review

Мақсаты: РО бойынша: бағдарламалық кодтың сапасын статикалық талдау және code review әдістерін қолдану дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: код сапасының проблемаларын анықтау, coding standards талаптарын тексеру және бағдарламалық кодты кәсіби сараптау дағдыларын игеру.

Әдістемелік нұсқаулық

1. Бағдарламалық кодты таңдаңыз.
2. Код құрылымын зерттеңіз.
3. Coding Standards анықтаңыз.
4. Static Analyzer таңдаңыз.
5. Құралды орнатыңыз.
6. Кодты тексеріңіз.
7. Warning тізімін алыңыз.
8. Error тізімін анықтаңыз.
9. Code Smell табыңыз.
10. Complexity есептеңіз.
11. Duplication анықтаңыз.
12. Security issue іздеңіз.

13. Code Review checklist жасаңыз.
14. Кодты қолмен тексеріңіз.
15. Проблемаларды басымдық бойынша жіктеңіз.
16. Кодты түзетіңіз.
17. Static Analysis қайталаңыз.
18. Нәтижелерді салыстырыңыз.
19. Quality Gate бағалаңыз.
20. Есепті рәсімдеңіз.

Жабдықтау

SonarQube/SonarLint, Pylint/Ruff, VS Code, GitHub, Python/Java/C#.

Тапсырмалар

1. Код жобасын таңдаңыз.
2. Coding Standards анықтаңыз.
3. Pylint/Ruff орнатыңыз.
4. Кодқа Static Analysis жүргізіңіз.
5. Warning тізімін алыңыз.
6. Error тізімін алыңыз.
7. Code Smell анықтаңыз.
8. Duplicate Code анықтаңыз.
9. Complexity талдаңыз.
10. Security issue іздеңіз.
11. Code Review Checklist жасаңыз.
12. Кодты қолмен тексеріңіз.
13. 10 кемшілікті анықтаңыз.
14. Кемшіліктерді Severity бойынша жіктеңіз.
15. Кодты түзетіңіз.
16. Анализді қайта жүргізіңіз.
17. Нәтижелерді салыстырыңыз.
18. Quality Gate нәтижесін анықтаңыз.
19. Код сапасының метрикаларын кестеге енгізіңіз.
20. Қорытынды есеп дайындаңыз.

Қорытынды

Кодтың бастапқы және түзетілген нұсқаларын салыстыратын Static Analysis және Code Review есебін жасаңыз.

Бақылау сұрақтары

1. Static Code Analysis дегеніміз не?
2. Code Review дегеніміз не?
3. Code Smell дегеніміз не?
4. SonarQube не үшін қолданылады?
5. Pylint не үшін қажет?
6. Ruff дегеніміз не?
7. Quality Gate дегеніміз не?
8. Coding Standard дегеніміз не?
9. Code Duplication дегеніміз не?

10. Cyclomatic Complexity дегеніміз не?
11. Static Analysis артықшылығы қандай?
12. Оның кемшіліктері қандай?
13. Security issue дегеніміз не?
14. Manual Code Review не үшін қажет?
15. Automated Code Review дегеніміз не?
16. Warning пен Error айырмашылығы қандай?
17. Code Review Checklist не үшін қажет?
18. Static Analysis Testing-ті алмастыра ала ма?
19. CI/CD-ге Static Analysis қалай енгізіледі?
20. Код сапасын қандай метрикалармен бағалауға болады?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20
Талдау сапасы	10
Есеп	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Martin R. C. *Clean Code.*
2. Fowler M. *Refactoring.*
3. SonarQube Documentation.
4. Pylint Documentation.
5. Ruff Documentation.

ПРАКТИКАЛЫҚ ЖҰМЫС №9

Тақырып: REST API тестілеу

Мақсаты:РО бойынша: REST API қызметтерін тестілеу, HTTP сұраныстары мен жауаптарын талдау дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: API интерфейстерінің функционалдығын, деректер алмасуын және қате жағдайларын тестілеу дағдыларын игеру.

Әдістемелік нұсқаулық

1. API сервисін таңдаңыз.
2. API құжаттамасын зерттеңіз.
3. Endpoint тізімін анықтаңыз.
4. HTTP әдістерін анықтаңыз.
5. GET сұранысын орындаңыз.
6. POST сұранысын орындаңыз.
7. PUT/PATCH сұранысын орындаңыз.

8. DELETE сұранысын орындаңыз.
9. Status Code талдаңыз.
10. Response Body тексеріңіз.
11. Headers тексеріңіз.
12. Authentication тексеріңіз.
13. Invalid Data қолданыңыз.
14. Boundary Data қолданыңыз.
15. Response Time өлшеңіз.
16. Test Collection жасаңыз.
17. Assertions қосыңыз.
18. Тесттерді автоматты іске қосыңыз.
19. Failed Request талдаңыз.
20. Есепті рәсімдеңіз.

Жабдықтау

Postman, Swagger/OpenAPI, браузер, REST API, Git.

Тапсырмалар

1. REST API таңдаңыз.
2. API documentation зерттеңіз.
3. Endpoint тізімін жасаңыз.
4. GET сұранысын орындаңыз.
5. POST сұранысын орындаңыз.
6. PUT сұранысын орындаңыз.
7. PATCH сұранысын орындаңыз.
8. DELETE сұранысын орындаңыз.
9. Status Code тексеріңіз.
10. Response Body тексеріңіз.
11. Header тексеріңіз.
12. Authentication тексеріңіз.
13. Қате параметр жіберіңіз.
14. Бос параметр жіберіңіз.
15. Boundary мәндерін жіберіңіз.
16. Response Time өлшеңіз.
17. Postman Collection жасаңыз.
18. Assertions қосыңыз.
19. Collection Runner арқылы тест орындаңыз.
20. API тестілеу есебін жасаңыз.

Қорытынды

REST API үшін кемінде **20 тестілік сценарийден** тұратын Postman Collection ұсыныңыз.

Бақылау сұрақтары

1. API дегеніміз не?
2. REST дегеніміз не?
3. Endpoint дегеніміз не?
4. GET не үшін қолданылады?
5. POST не үшін қолданылады?

6. PUT пен PATCH айырмашылығы қандай?
7. DELETE не үшін қажет?
8. HTTP Status Code дегеніміз не?
9. 200, 201, 400, 401, 403, 404, 500 кодтарын түсіндіріңіз.
10. JSON дегеніміз не?
11. Request Header дегеніміз не?
12. Response Header дегеніміз не?
13. Authentication дегеніміз не?
14. Authorization дегеніміз не?
15. API Testing дегеніміз не?
16. Postman не үшін қолданылады?
17. Assertion дегеніміз не?
18. API тестілерін автоматтандыруға бола ма?
19. Negative API Testing дегеніміз не?
20. API тестілеу не үшін маңызды?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20
Postman Collection	10
Есеп	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. ISTQB Foundation Level.
2. Postman Learning Center.
3. REST API Documentation.
4. OpenAPI Specification.
5. OWASP API Security Top 10.

ПРАКТИКАЛЫҚ ЖҰМЫС №10

Тақырып: Web-қосымшаларды тестілеу

Мақсаты: РО бойынша: web-қосымшалардың интерфейсін, функционалдығын және пайдаланушы сценарийлерін тестілеу дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: браузерлік тестілеу, UI автоматтандыру және cross-browser testing дағдыларын игеру.

Әдістемелік нұсқаулық

1. Web-қосымшаны таңдаңыз.
2. UI элементтерін анықтаңыз.
3. Пайдаланушы сценарийін жасаңыз.

4. Login функциясын тексеріңіз.
5. Registration тексеріңіз.
6. Form Validation тексеріңіз.
7. Navigation тексеріңіз.
8. Search тексеріңіз.
9. Responsive Design тексеріңіз.
10. Browser Compatibility тексеріңіз.
11. Selenium/Playwright орнатыңыз.
12. Locator анықтаңыз.
13. Автоматтандырылған тест жасаңыз.
14. Тестті іске қосыңыз.
15. Screenshot сақтаңыз.
16. Failed Test талдаңыз.
17. Кодты түзетіңіз.
18. Retest орындаңыз.
19. Regression Test орындаңыз.
20. Есепті рәсімдеңіз.

Жабдықтау

Chrome/Firefox/Edge, Selenium немесе Playwright, Python/JavaScript, VS Code, Git.

Тапсырмалар

1. Web-қосымшаны таңдаңыз.
2. 10 UI элементін анықтаңыз.
3. Login тестін жасаңыз.
4. Registration тестін жасаңыз.
5. Form Validation тестін жасаңыз.
6. Search тестін жасаңыз.
7. Navigation тестін жасаңыз.
8. Logout тестін жасаңыз.
9. Invalid Input тестін жасаңыз.
10. Responsive тест жасаңыз.
11. Selenium/Playwright орнатыңыз.
12. Locator жасаңыз.
13. Автоматтандырылған Login тестін жазыңыз.
14. Автоматтандырылған Search тестін жазыңыз.
15. Screenshot сақтаңыз.
16. Тест нәтижесін тіркеңіз.
17. Failed тестті талдаңыз.
18. Қатені түзетіңіз.
19. Regression тест орындаңыз.
20. Web Testing есебін жасаңыз.

Қорытынды

Web-қосымша үшін manual және automated UI testing нәтижелерін салыстырыңыз.

Бақылау сұрақтары

1. Web Testing дегеніміз не?

2. UI Testing дегеніміз не?
3. Selenium дегеніміз не?
4. Playwright дегеніміз не?
5. Locator дегеніміз не?
6. XPath дегеніміз не?
7. CSS Selector дегеніміз не?
8. Cross-browser Testing дегеніміз не?
9. Responsive Testing дегеніміз не?
10. Form Validation дегеніміз не?
11. UI Automation артықшылығы қандай?
12. UI Automation кемшілігі қандай?
13. Screenshot не үшін сақталады?
14. End-to-End Testing дегеніміз не?
15. Page Object Model дегеніміз не?
16. Browser Compatibility не үшін тексеріледі?
17. UI тестілерін CI/CD-де орындауға бола ма?
18. Manual және Automated Testing айырмашылығы қандай?
19. Flaky Test дегеніміз не?
20. Web Testing кезінде қандай ақаулар жиі кездеседі?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20
Автоматтандыру	10
Есеп	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Selenium Documentation.
2. Playwright Documentation.
3. ISTQB Foundation Level.
4. Myers G. *The Art of Software Testing.*
5. Fowler M. *Refactoring.*

ПРАКТИКАЛЫҚ ЖҰМЫС №11

Тақырып: Performance және Load Testing

Мақсаты:РО бойынша: бағдарламалық жүйенің өнімділігін, жауап беру уақытын және жүктеме кезіндегі тұрақтылығын бағалау дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: performance testing құралдарын қолдану, жүктемелік тест нәтижелерін талдау және өнімділік мәселелерін анықтау дағдыларын игеру.

Әдістемелік нұсқаулық

1. Тест объектісін таңдаңыз.
2. Performance талаптарын анықтаңыз.
3. Response Time өлшеңіз.
4. Throughput анықтаңыз.
5. Concurrent Users белгілеңіз.
6. Baseline Test орындаңыз.
7. Load Test жасаңыз.
8. Stress Test жасаңыз.
9. Spike Test жасаңыз.
10. Endurance Test жоспарлаңыз.
11. JMeter/Locust орнатыңыз.
12. Test Scenario жасаңыз.
13. Virtual Users белгілеңіз.
14. Load Profile жасаңыз.
15. Тестті іске қосыңыз.
16. Нәтижелерді сақтаңыз.
17. Bottleneck анықтаңыз.
18. Нәтижелерді салыстырыңыз.
19. Ұсыныстар жасаңыз.
20. Есепті рәсімдеңіз.

Жабдықтау

Apache JMeter немесе Locust, Python, браузер, Postman, Excel.

Тапсырмалар

1. Performance объектісін таңдаңыз.
2. Performance талаптарын анықтаңыз.
3. Baseline Response Time өлшеңіз.
4. Throughput анықтаңыз.
5. 10 пайдаланушы жүктемесін тексеріңіз.
6. 25 пайдаланушы жүктемесін тексеріңіз.
7. 50 пайдаланушы жүктемесін тексеріңіз.
8. 100 пайдаланушы жүктемесін жоспарлаңыз.
9. Load Test орындаңыз.
10. Stress Test орындаңыз.
11. Spike Test жасаңыз.
12. Endurance Test жоспарын жасаңыз.
13. JMeter/Locust сценарийін құрыңыз.
14. Virtual Users параметрін орнатыңыз.
15. Response Time тіркеңіз.
16. Error Rate есептеңіз.
17. Throughput салыстырыңыз.
18. Bottleneck анықтаңыз.
19. Performance графигін жасаңыз.
20. Қорытынды есеп жасаңыз.

Қорытынды

Қосымшаның жүктеме кезіндегі өнімділік көрсеткіштерін кесте және график түрінде ұсыныңыз.

Бақылау сұрақтары

1. Performance Testing дегеніміз не?
2. Load Testing дегеніміз не?
3. Stress Testing дегеніміз не?
4. Spike Testing дегеніміз не?
5. Endurance Testing дегеніміз не?
6. Response Time дегеніміз не?
7. Throughput дегеніміз не?
8. Concurrent User дегеніміз не?
9. Bottleneck дегеніміз не?
10. Error Rate дегеніміз не?
11. Baseline дегеніміз не?
12. Load Profile дегеніміз не?
13. JMeter не үшін қолданылады?
14. Locust дегеніміз не?
15. Performance Testing не үшін қажет?
16. Load және Stress Testing айырмашылығы қандай?
17. Performance метрикалары қандай?
18. Жүктеме артқанда Response Time неге өседі?
19. Performance проблемасын қалай анықтауға болады?
20. Performance Testing нәтижесі қалай бағаланады?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20
Тест нәтижелері	10
Есеп	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Apache JMeter Documentation.
2. Locust Documentation.
3. ISTQB Performance Testing.
4. Humble J., Farley D. *Continuous Delivery.*
5. Forsgren N. et al. *Accelerate.*

ПРАКТИКАЛЫҚ ЖҰМЫС №12

Тақырып: Security Testing және OWASP Top 10

Мақсаты:РО бойынша: web-қосымшалардың негізгі қауіпсіздік осалдықтарын анықтау және қауіпсіздік тестілерін жүргізу дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: қауіпсіздік тәуекелдерін бағалау, OWASP ұсынымдарын қолдану және осалдықтарды құжаттау дағдыларын игеру.

Әдістемелік нұсқаулық

1. Тек оқу мақсатындағы тест ортасын таңдаңыз.
2. Қауіпсіздік талаптарын анықтаңыз.
3. Authentication тексеріңіз.
4. Authorization тексеріңіз.
5. Input Validation тексеріңіз.
6. Access Control тексеріңіз.
7. Session Management тексеріңіз.
8. Injection тәуекелін зерттеңіз.
9. XSS тәуекелін зерттеңіз.
10. Security Headers тексеріңіз.
11. OWASP ZAP орнатыңыз.
12. Passive Scan орындаңыз.
13. Active Scan тек рұқсат етілген ортада орындаңыз.
14. Vulnerability тізімін алыңыз.
15. Severity анықтаңыз.
16. False Positive тексеріңіз.
17. Қауіпсіздік ұсыныстарын жасаңыз.
18. Осалдықтарды түзетіңіз.
19. Қайта сканерлеңіз.
20. Security Report дайындаңыз.

Жабдықтау

OWASP ZAP, Burp Suite Community Edition, браузер, жергілікті оқу web-қосымшасы, OWASP материалдары.

Тапсырмалар

1. Оқу web-қосымшасын таңдаңыз.
2. Security Requirements жасаңыз.
3. Authentication тестін жасаңыз.
4. Authorization тестін жасаңыз.
5. Input Validation тестін жасаңыз.
6. Session Management тексеріңіз.
7. Security Headers тексеріңіз.
8. Injection тәуекелін зерттеңіз.
9. XSS тәуекелін зерттеңіз.
10. Access Control тексеріңіз.
11. OWASP ZAP іске қосыңыз.
12. Passive Scan орындаңыз.
13. Scan нәтижелерін сақтаңыз.
14. Vulnerability тізімін жасаңыз.
15. Severity жіктеңіз.
16. False Positive анықтаңыз.

17. Екі осалдық бойынша түзету ұсыныңыз.
18. Түзетілген нұсқаны қайта тестілеңіз.
19. Before/After нәтижесін салыстырыңыз.
20. Security Report дайындаңыз.

Қорытынды

Анықталған осалдықтарды **Vulnerability → Risk → Severity → Recommendation → Retest** тізбегі бойынша сипаттаңыз.

Бақылау сұрақтары

1. Security Testing дегеніміз не?
2. OWASP дегеніміз не?
3. OWASP Top 10 не үшін қажет?
4. Authentication дегеніміз не?
5. Authorization дегеніміз не?
6. Access Control дегеніміз не?
7. Injection дегеніміз не?
8. XSS дегеніміз не?
9. CSRF дегеніміз не?
10. Security Headers дегеніміз не?
11. Vulnerability дегеніміз не?
12. Risk дегеніміз не?
13. Severity дегеніміз не?
14. False Positive дегеніміз не?
15. OWASP ZAP не үшін қолданылады?
16. Burp Suite не үшін қолданылады?
17. Passive Scan дегеніміз не?
18. Active Scan дегеніміз не?
19. Security Testing неге арнайы тест ортасында орындалуы тиіс?
20. Қауіпсіздік осалдығы түзетілгеннен кейін не істеу керек?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20
Security Analysis	10
Есеп	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. OWASP Web Security Testing Guide.
2. OWASP Top 10.
3. OWASP API Security Top 10.
4. OWASP ZAP Documentation.

5. Burp Suite Documentation.
6. ISTQB Security Tester.

ПРАКТИКАЛЫҚ ЖҰМЫС №13

Тақырып: Regression Testing және Test Automation

Мақсаты:РО бойынша: бағдарламалық жүйеге өзгерістер енгізілгеннен кейін бұрынғы функционалдардың жұмысын қайта тексеру және regression тестілерін автоматтандыру дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: regression test suite қалыптастыру, автоматтандыру және тест нәтижелерін талдау дағдыларын игеру.

Әдістемелік нұсқаулық

1. Бағдарламаның бастапқы нұсқасын таңдаңыз.
2. Негізгі функционалдарды анықтаңыз.
3. Regression Test Case таңдаңыз.
4. Smoke Test жасаңыз.
5. Test Suite құрыңыз.
6. Автоматтандырылатын тестілерді таңдаңыз.
7. Automation Framework таңдаңыз.
8. Test Script жазыңыз.
9. Test Data дайындаңыз.
10. Тестілерді орындаңыз.
11. Failed Test анықтаңыз.
12. Бағдарламаға өзгеріс енгізіңіз.
13. Regression Test қайта орындаңыз.
14. Нәтижелерді салыстырыңыз.
15. Flaky Test анықтаңыз.
16. Test Suite оңтайландырыңыз.
17. Test Coverage анықтаңыз.
18. Regression Test автоматтандыру деңгейін есептеңіз.
19. Нәтижелерді талдаңыз.
20. Есепті рәсімдеңіз.

Жабдықтау

Python, pytest, Selenium/Playwright, Git, GitHub, IDE.

Тапсырмалар

1. Бастапқы қосымшаны таңдаңыз.
2. 20 негізгі функцияны анықтаңыз.
3. Smoke Test жасаңыз.
4. Regression Test Case жасаңыз.
5. Test Suite құрыңыз.
6. Автоматтандыруға болатын тестілерді таңдаңыз.
7. 10 автоматтандырылған тест жасаңыз.
8. Test Data дайындаңыз.
9. Тестілерді орындаңыз.

10. Нәтижелерді тіркеңіз.
11. Failed Test анықтаңыз.
12. Бағдарламаға өзгеріс енгізіңіз.
13. Regression Test қайта іске қосыңыз.
14. Нәтижелерді салыстырыңыз.
15. Broken Function анықтаңыз.
16. Flaky Test іздеңіз.
17. Test Suite оңтайландырыңыз.
18. Test Coverage есептеңіз.
19. Automation Rate есептеңіз.
20. Regression Testing есебін жасаңыз.

Қорытынды

Бағдарлама өзгергенге дейінгі және кейінгі тест нәтижелерін салыстырып, regression testing тиімділігін бағалаңыз.

Бақылау сұрақтары

1. Regression Testing дегеніміз не?
2. Smoke Testing дегеніміз не?
3. Sanity Testing дегеніміз не?
4. Regression Suite дегеніміз не?
5. Test Automation дегеніміз не?
6. Automation Framework дегеніміз не?
7. Regression тесті қашан орындалады?
8. Smoke және Regression Testing айырмашылығы қандай?
9. Flaky Test дегеніміз не?
10. Test Coverage дегеніміз не?
11. Automation Rate дегеніміз не?
12. Regression тестілерін неге автоматтандыру қажет?
13. Қандай тестілерді автоматтандырмаған дұрыс?
14. Test Suite қалай оңтайландырылады?
15. Test Data қандай рөл атқарады?
16. Regression Testing CI/CD-де қалай орындалады?
17. Failed Regression Test нені білдіреді?
18. Regression Testing қаншалықты жиі орындалуы керек?
19. Regression Testing пен Retesting айырмашылығы қандай?
20. Regression Suite сапасын қалай бағалауға болады?

Бағалау критерийлері

Критерий	**Балл**
1–5	20
6–10	20
11–15	20
16–20	20
Automation	10
Есеп	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Meszaros G. *xUnit Test Patterns.*
2. Selenium Documentation.
3. Playwright Documentation.
4. pytest Documentation.
5. ISTQB Foundation Level.

ПРАКТИКАЛЫҚ ЖҰМЫС №14

Тақырып: CI/CD және Continuous Testing

Мақсаты: РО бойынша: автоматтандырылған тестілеуді CI/CD процесіне енгізу және бағдарламалық жобаның сапасын үздіксіз бақылау дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: Git, CI/CD pipeline, automated testing және quality gate механизмдерін қолдану дағдыларын игеру.

Әдістемелік нұсқаулық

1. Git репозиторийін жасаңыз.
2. Бағдарламалық жобаны орналастырыңыз.
3. Unit Test қосыңыз.
4. Dependency файл жасаңыз.
5. CI платформасын таңдаңыз.
6. Pipeline файлын жасаңыз.
7. Build кезеңін анықтаңыз.
8. Test кезеңін анықтаңыз.
9. Static Analysis кезеңін қосыңыз.
10. Test Report жасаңыз.
11. Pipeline іске қосыңыз.
12. Failed Pipeline талдаңыз.
13. Қатені түзетіңіз.
14. Pipeline қайта іске қосыңыз.
15. Quality Gate анықтаңыз.
16. Coverage талабын орнатыңыз.
17. Artifact сақтаңыз.
18. Notification механизмін қарастырыңыз.
19. Continuous Testing процесін талдаңыз.
20. CI/CD есебін рәсімдеңіз.

Жабдықтау

Git, GitHub/GitLab, GitHub Actions/GitLab CI, Python, pytest, SonarQube/SonarCloud.

Тапсырмалар

1. Git репозиторий жасаңыз.
2. Жобаны репозиторийге жүктеңіз.
3. Unit Test қосыңыз.
4. requirements.txt жасаңыз.
5. GitHub Actions таңдаңыз.

6. Workflow файлын жасаңыз.
7. Build Stage құрыңыз.
8. Test Stage құрыңыз.
9. Static Analysis Stage құрыңыз.
10. Coverage есептеу қосыңыз.
11. Pipeline іске қосыңыз.
12. Pipeline нәтижесін зерттеңіз.
13. Әдейі тест қатесін жасаңыз.
14. Failed Pipeline анықтаңыз.
15. Қатені түзетіңіз.
16. Pipeline қайта іске қосыңыз.
17. Quality Gate жасаңыз.
18. Test Artifact сақтаңыз.
19. Continuous Testing процесін схема түрінде көрсетіңіз.
20. CI/CD есебін дайындаңыз.

Қорытынды

Git → Build → Test → Analysis → Quality Gate → Deploy логикасы бойынша CI/CD pipeline көрсетіңіз.

Бақылау сұрақтары

1. CI дегеніміз не?
2. CD дегеніміз не?
3. CI/CD не үшін қажет?
4. Continuous Testing дегеніміз не?
5. Pipeline дегеніміз не?
6. Stage дегеніміз не?
7. GitHub Actions дегеніміз не?
8. GitLab CI дегеніміз не?
9. Workflow дегеніміз не?
10. Build Stage не үшін қажет?
11. Test Stage не үшін қажет?
12. Quality Gate дегеніміз не?
13. Artifact дегеніміз не?
14. CI кезінде Unit Test неге орындалады?
15. Pipeline Failure дегеніміз не?
16. Continuous Integration артықшылығы қандай?
17. Automated Testing CI/CD-ге қалай енгізіледі?
18. Static Analysis pipeline-да не үшін орындалады?
19. Test Coverage Quality Gate ретінде қолданылуы мүмкін бе?
20. Continuous Testing пен дәстүрлі Testing айырмашылығы қандай?

Бағалау критерийлері

Критерий	Балл
1–5	20
6–10	20
11–15	20
16–20	20

Критерий	Балл
Pipeline	10
Есеп	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Humble J., Farley D. *Continuous Delivery.*
2. Kim G. et al. *The DevOps Handbook.*
3. Forsgren N. et al. *Accelerate.*
4. GitHub Actions Documentation.
5. GitLab CI/CD Documentation.
6. pytest Documentation.

ПРАКТИКАЛЫҚ ЖҰМЫС №15

Тақырып: Бағдарламалық қосымшаны кешенді тестілеу және сапасын бағалау

Мақсаты:РО бойынша: бағдарламалық қосымшаны функционалдық, интеграциялық, өнімділік, қауіпсіздік және автоматтандырылған тестілеу нәтижелері негізінде кешенді бағалау дағдыларын қалыптастыру.

Кәсіби құзыреттер бойынша: бағдарламалық өнімнің сапасын кешенді талдау, тест нәтижелерін біріктіру, ақауларды бағалау және бағдарламалық өнімді пайдалануға дайындығы туралы кәсіби қорытынды жасау дағдыларын игеру.

Әдістемелік нұсқаулық

1. Бағдарламалық жобаны таңдаңыз.
2. Функционалдық талаптарды талдаңыз.
3. Test Plan жасаңыз.
4. Test Case жинағын жасаңыз.
5. Unit Test орындаңыз.
6. Integration Test орындаңыз.
7. System Test орындаңыз.
8. API Test орындаңыз.
9. UI Test орындаңыз.
10. Regression Test орындаңыз.
11. Performance Test орындаңыз.
12. Security Test орындаңыз.
13. Static Analysis орындаңыз.
14. Defect Report жасаңыз.
15. Severity анықтаңыз.
16. Test Coverage есептеңіз.
17. Quality Metrics есептеңіз.
18. Қалдық тәуекелдерді анықтаңыз.
19. Release Readiness бағалаңыз.
20. Қорытынды тестілеу есебін рәсімдеңіз.

Жабдықтау

Python/Java/C#, pytest/JUnit, Postman, Selenium/Playwright, JMeter/Locust, OWASP ZAP, SonarQube, Git/GitHub, GitHub Actions.

Тапсырмалар

1. Кешенді тестілеу үшін қосымша таңдаңыз.
2. Функционалдық талаптарды анықтаңыз.
3. Test Plan жасаңыз.
4. 20 Test Case дайындаңыз.
5. Unit Testing орындаңыз.
6. Integration Testing орындаңыз.
7. System Testing орындаңыз.
8. API Testing орындаңыз.
9. UI Testing орындаңыз.
10. Regression Testing орындаңыз.
11. Performance Testing орындаңыз.
12. Security Testing орындаңыз.
13. Static Code Analysis жүргізіңіз.
14. Анықталған Defect тізімін жасаңыз.
15. Defect Severity анықтаңыз.
16. Test Coverage есептеңіз.
17. Pass/Fail Rate есептеңіз.
18. Қалдық тәуекелдерді бағалаңыз.
19. Release Readiness шешімін қабылдаңыз.
20. Қорытынды Quality Assurance Report дайындаңыз.

Қорытынды

Қорытынды есепте:

- тестіленген функционалдар;
- орындалған тестілер саны;
- Passed/Failed тестілер;
- анықталған ақаулар;
- Critical/High/Medium/Low ақаулар;
- Test Coverage;
- Performance нәтижелері;
- Security нәтижелері;
- Static Analysis нәтижелері;
- Regression нәтижелері;
- қалдық тәуекелдер;
- Release Readiness

көрсетілуі тиіс.

Бақылау сұрақтары

1. Кешенді тестілеу дегеніміз не?
2. Quality Assurance дегеніміз не?
3. Quality Control дегеніміз не?
4. QA мен QC айырмашылығы қандай?
5. Бағдарламалық өнім сапасы қалай бағаланады?

6. Test Coverage дегеніміз не?
7. Pass Rate дегеніміз не?
8. Fail Rate дегеніміз не?
9. Defect Density дегеніміз не?
10. Severity дегеніміз не?
11. Priority дегеніміз не?
12. Critical Defect дегеніміз не?
13. Residual Risk дегеніміз не?
14. Release Readiness дегеніміз не?
15. Қашан бағдарламаны release жасауға болады?
16. Қандай тестілер release алдында міндетті?
17. Automation тестілеудің қандай артықшылығы бар?
18. Performance және Security Testing не үшін қажет?
19. Static Analysis нәтижесі сапаны бағалауға қалай әсер етеді?
20. Кешенді тестілеу нәтижесінде қандай шешім қабылданады?

Бағалау критерийлері

Критерий	Балл
1–5 тапсырмалар	20
6–10 тапсырмалар	20
11–15 тапсырмалар	20
16–20 тапсырмалар	20
Кешенді талдау	10
Қорытынды есеп және қорғау	10
Барлығы	**100**

Ұсынылатын әдебиеттер

1. Myers G. J., Sandler C., Badgett T. *The Art of Software Testing.*
2. Ammann P., Offutt J. *Introduction to Software Testing.*
3. Graham D. et al. *Foundations of Software Testing.*
4. Meszaros G. *xUnit Test Patterns.*
5. Humble J., Farley D. *Continuous Delivery.*
6. Kim G. et al. *The DevOps Handbook.*
7. Forsgren N. et al. *Accelerate.*
8. Martin R. C. *Clean Code.*
9. ISTQB Certified Tester Foundation Level.
10. ISO/IEC 25010.
11. OWASP Web Security Testing Guide.
12. pytest Documentation.
13. Selenium Documentation.
14. Playwright Documentation.
15. Postman Learning Center.
16. SonarQube Documentation.
17. GitHub Actions Documentation.
18. Apache JMeter Documentation.
19. Locust Documentation.
20. OWASP ZAP Documentation.